

**LAPORAN PRAKTIKUM
IMPLEMENTASI PERANGKAT LUNAK**

**MODUL 2
DESIGN PRINCIPLE (S.O.L.I.D)**

**DISUSUN OLEH :
BAGUS DWI PUTRA SETIAWAN - 2350081042**



**PROGRAM STUDI INFORMATIKA
FAKULTAS SAINS DAN INFORMATIKA
UNIVERSITAS JENDERAL ACHMAD YANI
TAHUN 2025**

DAFTAR ISI

DAFTAR ISI.....	i
DAFTAR GAMBAR	ii
BAB I. HASIL PRAKTIKUM.....	1
I.1 Latihan SRP (Single Responsibility Principle).	1
I.1.A Langkah Kerja / Source Code	1
I.1.B Analisis.....	4
I.2 Latihan OSP (Open / Close Principle).	4
I.2.A Langkah Kerja / Source Code	4
I.2.B Screenshoot	8
I.2.C Analisis.....	9
I.3 Latihan LSP (Liskov Substitution Principle).	9
I.3.A Langkah Kerja / Source Code	9
I.3.B Analisis.....	12
BAB II. TUGAS PRAKTIKUM	13
II.1 Tugas	13
BAB III. KESIMPULAN.....	15

DAFTAR GAMBAR

Gambar 1. 1 Hasil Latihan OCP	8
-------------------------------------	---

BAB I. HASIL PRAKTIKUM

I.1 Latihan SRP (Single Responsibility Principle).

I.1.A Langkah Kerja / Source Code

Code Tanpa Prinsip SRP

Class Order

```
class Order {  
    void calculateTotalSum() {  
        /*...*/  
    }  
  
    void.getItems() {  
        /*...*/  
    }  
  
    void getItemCount() {  
        /*...*/  
    }  
  
    void addItem(Item item) {  
        /*...*/  
    }  
  
    void deleteItem(Item item) {  
        /*...*/  
    }  
  
    void printOrder() {  
        /*...*/  
    }  
}
```

```
}

void showOrder() {
    /*...*/
}

void getDailyHistory() {
    /*...*/
}

void getMonthlyHistory() {
    /*...*/
}
}
```

Code Dengan Prinsip SRP

Class Order

```
class Order {
    void calculateTotalSum() {
    }

    void.getItems() {
    }

    void.getItemCount() {
    }

    void addItem(Item item) {
    }
}
```

```
        void deleteItem(Item item) {  
            }  
    }  
}
```

Class OrderHistory

```
class OrderHistory {  
    void getDailyHistory() {  
    }  
  
    void getMonthlyHistory() {  
    }  
}
```

Class OrderViewer

```
class OrderViewer {  
    void printOrder(Order order) {  
    }  
  
    void showOrder(Order order) {  
    }  
}
```

Class Main

```
class Main {  
    public static void main(String[] args) {  
        Item item = new Item();  
    }  
}
```

```
        Order order = new Order();
        order.addItem(item);

        OrderHistory history = new OrderHistory();
        history.getDailyHistory();

        OrderViewer viewer = new OrderViewer();
        viewer.printOrder(order);
    }
}
```

I.1.B Analisis

Hasil analisis yang saya dapat pada Latihan ini yaitu konsep Single Responsibility Principle (SRP) diterapkan untuk memisahkan tanggung jawab kelas agar setiap kelas hanya memiliki satu alasan untuk berubah. Pada kode awal, seluruh fungsi seperti pengelolaan pesanan, riwayat, dan tampilan digabungkan dalam satu kelas Order. Hal ini melanggar SRP karena kelas tersebut menangani lebih dari satu tanggung jawab.

Setelah diterapkan SRP, tanggung jawab dibagi menjadi tiga kelas terpisah yaitu Order hanya mengelola data pesanan, OrderHistory menangani riwayat transaksi, dan OrderViewer mengatur tampilan pesanan. Pembagian ini membuat kode lebih mudah dirawat, diuji, dan dikembangkan tanpa saling memengaruhi fungsionalitas lain di dalam sistem.

I.2 Latihan OSP (Open / Close Principle).

I.2.A Langkah Kerja / Source Code

Code Tanpa Prinsip OSP

Class Cinema

```
class Cinema {  
    public Double price;  
}
```

Class StandardCinema

```
class StandardCinema extends Cinema {  
    public StandardCinema(double price) {  
        this.price = price;  
    }  
}
```

Class DeluxeCinema

```
class DeluxeCinema extends Cinema {  
    public DeluxeCinema(double price) {  
        this.price = price;  
    }  
}
```

Class PremiumCinema

```
class PremiumCinema extends Cinema {  
    public PremiumCinema(double price) {  
        this.price = price;  
    }  
}
```

Class CinemaCalculations


```

class CinemaCalculations {
    public Double calculateAdminFee(Cinema cinema)
    {
        if (cinema instanceof StandardCinema) {
            return ((StandardCinema) cinema).price
* 10 / 100;
        } else if (cinema instanceof DeluxeCinema)
        {
            return ((DeluxeCinema) cinema).price *
12 / 100;
        } else if (cinema instanceof
PremiumCinema) {
            return ((PremiumCinema) cinema).price
* 20 / 100;
        } else {
            return 0.0;
        }
    }
}

```

Code Dengan Prinsip OSP

Class Cinema

```

abstract class Cinema {
    public Double price;
    abstract Double calculateAdminFee();
}

```

Class StandardCinema

```
class StandardCinema extends Cinema {  
    public StandardCinema(Double price) {  
        this.price = price;  
    }  
  
    @Override  
    Double calculateAdminFee() {  
        return price * 10 / 100;  
    }  
}
```

Class DeluxeCinema

```
class DeluxeCinema extends Cinema {  
    public DeluxeCinema(Double price) {  
        this.price = price;  
    }  
  
    @Override  
    Double calculateAdminFee() {  
        return price * 12 / 100;  
    }  
}
```

Class PremiumCinema

```
class PremiumCinema extends Cinema {  
    public PremiumCinema(Double price) {  
        this.price = price;  
    }  
}
```

```

@Override
Double calculateAdminFee() {
    return price * 20 / 100;
}
}

```

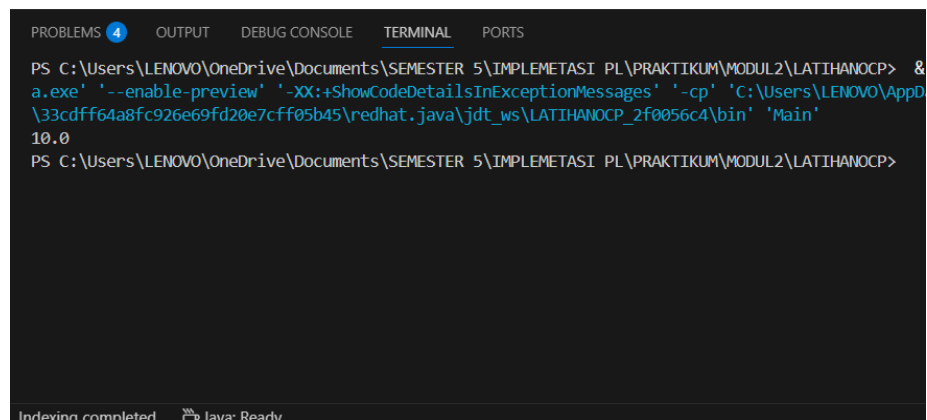
Class Main

```

class Main {
    public static void main(String[] args) {
        StandardCinema standardCinema = new
StandardCinema(100.0);
        Double adminFee =
standardCinema.calculateAdminFee();
        System.out.println(adminFee);
    }
}

```

I.2.B Screenshoot



```

PROBLEMS 4 OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\LENOVO\OneDrive\Documents\SEMESTER 5\IMPLEMETASI PL\PRAKTIKUM\MODUL2\LATIHANOC2f0056c4> java -enable-preview -XX:+ShowCodeDetailsInExceptionMessages -cp 'C:\Users\LENOVO\AppData\Local\Microsoft\Windows\Apps\Java\jdt_ws\LATIHANOC2f0056c4\bin' Main
10.0
PS C:\Users\LENOVO\OneDrive\Documents\SEMESTER 5\IMPLEMETASI PL\PRAKTIKUM\MODUL2\LATIHANOC2f0056c4>

```

Gambar 1. 1 Hasil Latihan OCP

I.2.C Analisis

Hasil analisis yang saya dapat pada Latihan ini yaitu menerapkan Open / Closed Principle (OCP) yang berarti kelas harus terbuka untuk ekstensi namun tertutup untuk modifikasi. Pada kode awal, kelas CinemaCalculations harus diubah setiap kali tipe bioskop baru ditambahkan karena menggunakan banyak if-else berdasarkan tipe objek. Hal ini tidak sesuai dengan prinsip OCP karena menyebabkan ketergantungan yang tinggi antar kelas.

Setelah perbaikan setiap jenis bioskop seperti StandardCinema, DeluxeCinema, dan PremiumCinema memiliki metode calculateAdminFee() sendiri yang di-override dari kelas abstrak Cinema. Dengan cara ini, penambahan tipe bioskop baru cukup dilakukan dengan membuat subclass baru tanpa mengubah kode yang sudah ada, sehingga menjadi lebih fleksibel dan mudah diperluas.

I.3 Latihan LSP (Liskov Substitution Principle).

I.3.A Langkah Kerja / Source Code

Code Tanpa Prinsip LSP

Class Product

```
abstract class Product {  
    abstract String getName();  
    abstract Date getExpiredDate();  
  
    public void getProductInfo() {  
        // some magic code  
    }  
}
```

Class Vegetable

```
class Vegetable extends Product {  
  
    @Override  
    String getName() {  
        return "Broccoli";  
    }  
  
    @Override  
    Date getExpiredDate() {  
        return new Date();  
    }  
}
```

Class Smartphone

```
class Smartphone extends Product {  
  
    @Override  
    String setName() {  
        return "Samsung S10+ Limited Edition";  
    }  
  
    @Override  
    Date setExpiredDate() {  
        return new Date(); // ???????  
    }  
}
```

Code Dengan Prinsip LSP

Class Product

```
abstract class Product {  
    abstract String getName();  
  
    /**  
     * Function to get all of information about  
product  
     */  
    public void getProductInfo() {  
        // some magic code  
    }  
}
```

Class FoodProduct

```
import java.util.Date;  
  
abstract class FoodProduct extends Product {  
    abstract Date getExpiredDate();  
}
```

Class Vegetable

```
import java.util.Date;  
  
class Vegetable extends FoodProduct {  
    @Override  
    String getName() {  
        return "Broccoli";  
    }  
}
```

```
@Override
    Date getExpiredDate() {
        return new Date();
    }
}
```

Class Smartphone

```
class Smartphone extends Product {
    @Override
    String getName() {
        return "Samsung S10+ Limited Edition";
    }
}
```

I.3.B Analisis

Hasil analisis yang saya dapat pada Latihan ini yaitu menerapkan Liskov Substitution Principle (LSP) yang menyatakan bahwa objek turunan harus dapat menggantikan objek induknya tanpa mengubah perilaku program. Pada kode awal, kelas Smartphone melanggar LSP karena tidak konsisten dengan metode yang diwarisi dari Product, yakni menggunakan setName() dan setExpiredDate() menggantikan getName() dan getExpiredDate(). Hal ini menyebabkan ketidaksesuaian antar kelas turunan.

Dalam versi yang telah diperbaiki, kelas FoodProduct ditambahkan sebagai kelas abstrak turunan dari Product untuk menampung produk yang memiliki tanggal kedaluwarsa. Kelas Vegetable mewarisi FoodProduct, sedangkan Smartphone tetap dari Product. Dengan struktur ini, setiap subclass mengikuti kontrak dari superclass-nya dan dapat saling menggantikan tanpa menimbulkan error atau perilaku tak terduga, sesuai prinsip LSP.

BAB II. TUGAS PRAKTIKUM

II.1 Tugas

1. Pada Latihan 3.2.1 Single Responsibility, Jelaskan alasan kenapa bentuk/design codenya harus diubah?

Jawab : desain kode harus diubah karena satu kelas Order memiliki banyak tanggung jawab sekaligus. Dengan memisahkannya menjadi beberapa kelas, setiap kelas hanya fokus pada satu tugas dan lebih mudah dikelola.

2. ada Latihan 3.2.2 Open/Closed, Jelaskan alasan kenapa bentuk/design codenya harus diubah?

Jawab : karena desain kode awal melanggar OCP sehingga harus diubah setiap kali tipe bioskop baru ditambahkan. Dengan membuat subclass sendiri untuk tiap tipe, kode bisa diperluas tanpa perlu dimodifikasi.

3. Buatlah Class CinemaMahasiswa yang biaya adminnya adalah 5% dari harga umum (5/100).

Jawab :

```
class CinemaMahasiswa extends Cinema {  
    public CinemaMahasiswa(Double price) {  
        this.price = price;  
    }  
  
    @Override  
    Double calculateAdminFee() {  
        return price * 5 / 100;  
    }  
}
```

4. Pada Latihan 3.2.3 Liskov Substitution, Jelaskan alasan kenapa bentuk/design codenya harus diubah?

Jawab : desain kode harus diubah karena subclass Smartphone tidak mengikuti struktur superclass Product. Setelah diperbaiki, semua subclass bisa saling menggantikan tanpa menyebabkan error.

BAB III. KESIMPULAN

Kesimpulan yang dapat saya ambil dalam pertemuan kedua pada Praktikum Implementasi Perangkat Lunak adalah dapat memahami dan menerapkan prinsip desain S.O.L.I.D dalam pemrograman berorientasi objek. Prinsip-prinsip ini membantu membuat kode yang lebih terstruktur, mudah dipelihara, dan fleksibel terhadap perubahan. Pada pertemuan ini difokuskan pada tiga prinsip utama yaitu SRP (Single Responsibility Principle), OCP (Open/Closed Principle), dan LSP (Liskov Substitution Principle). Melalui latihan dan tugas, dipelajari bagaimana setiap kelas harus memiliki satu tanggung jawab (SRP), dapat diperluas tanpa mengubah kode lama (OCP), serta menjaga agar subclass dapat menggantikan superclass tanpa menimbulkan kesalahan (LSP). Penerapan ketiga prinsip ini menjadikan desain sistem lebih efisien dan mudah dikembangkan.